

HYPERLOAD

Science-Grade Astrophotography Script Suite for Siril

Full Script & Function Reference · v2.0 · 2026

8

Suites

67+

Scripts

52

Core algorithms

Overview

HYPERLOAD is a comprehensive Python-based astrophotography processing suite designed to run within Siril, the free open-source astrophotography application. Every script provides a full PyQt6 graphical interface and science-grade algorithms comparable to or exceeding commercial tools such as PixInsight.

The suite is organised into eight thematic suites plus a collection of standalone specialist scripts. Each suite groups related processing steps into a single multi-tab window. All scripts can also be run independently. A central launcher (HYPERLOAD) provides a unified interface for managing, loading and running any script in the collection from a single window that integrates live system statistics and a file management dock.

Architecture

The folder structure is distribution-ready and location-independent. The user places the HYPERLOAD/ folder anywhere on disk and adds it to Siril's script directory preferences. Siril displays only hyperload.py in its menu; all child scripts in Scripts/ and Suites/ are invisible to Siril but fully accessible through the launcher.

- HYPERLOAD/ ← add this folder to Siril preferences
- hyperload.py ← Siril entry point
- hyperload_config.json ← auto-created, stores user paths
- assets/ logo.png
- Suites/ ← 8 merged multi-tab suite scripts
- Scripts/ ← all standalone scripts + launcher UI

Key Technical Principles

Lazy loading Scripts are imported only when the user clicks Run — not at startup. Cold-start time is under one second.

Process isolation Each script runs as a separate OS process, so a crash in one script cannot affect the launcher or other running scripts.

Memory-mapped FITS Large FITS files are opened with astropy's memmap=True, so only the tiles actually being processed are loaded into RAM.

Result caching Computationally expensive steps such as PSF heatmap generation and astrometric calibration cache their results based on a hash of the input data and parameters.

Shared equipment profiles The Equipment Manager provides a central JSON store for telescope, camera, filter and site parameters accessible to all scripts without re-entry.

Suite Reference

Image Processing Suite

`image_processing_suite.py`

The core image processing pipeline for deep-sky astrophotography. Provides non-linear stretching, background removal, mask construction, narrowband palette mapping, local normalisation, deconvolution and side-by-side A/B comparison. This suite is typically the first stop after stacking and covers everything from initial stretch to final aesthetic adjustments.

Wavelet Stretch

`wavelet_stretch.py`

Multi-scale wavelet-based non-linear stretching that preserves fine detail and colour accuracy simultaneously. Each wavelet decomposition band is stretched independently, allowing the user to control how faint nebulosity is lifted while keeping star cores and fine structure intact.

`[stretching] [non-linear] [wavelets] [luminance]`

Algorithm: à trous wavelet decomposition, per-band transfer curves, luminance-preserving colour mode

Dark Channel

`dark_channel.py`

Removes atmospheric haze, gradient vignetting and sky glow using the dark channel prior from computational photography. Adapted from He et al. 2009 for astrophotography, it estimates the transmission map from local minima across colour channels and uses it to invert the haze formation model.

`[haze] [vignette] [gradient] [sky-glow]`

Algorithm: Dark channel prior (He et al. 2009), atmospheric light estimation, guided filter refinement

Reference: He, Sun & Tang 2009, CVPR

Mask Builder

`mask_builder.py`

Constructs pixel-precise masks for stars, galaxies and nebulae using source detection and morphological operations. Masks can be grown, shrunk, feathered and combined with logical operators, enabling targeted processing of specific image regions without affecting others.

`[masking] [stars] [galaxy] [nebula] [morphology]`

Algorithm: DAOSTarFinder source detection, morphological dilation/erosion, Gaussian feathering

Narrowband Palette

`nb_palette.py`

Maps narrowband emission line data (H-alpha, OIII, SII) to RGB colour space using configurable palette presets. Supports the Hubble SHO palette, the HOO (natural colour) palette, HSO and fully custom channel weighting with per-channel curves.

`[narrowband] [Ha] [OIII] [SII] [SHO] [HOO] [palette]`

Algorithm: Linear channel matrix mixing, per-channel transfer curves, colour balance normalisation

Local Normalisation

`local_norm.py`

Removes large-scale background gradient variations by fitting and subtracting a smooth background model derived from source-free image regions. Particularly useful for mosaic panel matching and for correcting gradients introduced by light pollution or optical vignetting that survive flat-field correction.

`[background] [gradient] [mosaic] [normalisation]`

Algorithm: Median-filtered background estimation, polynomial or RBF surface fit, iterative sigma rejection

A/B Compare

[ab_compare.py](#)

Side-by-side comparator for evaluating two processing variants. Both panels share a synchronised zoom and pan state, allowing direct pixel-level comparison. Supports wipe mode (draggable dividing line), blink mode and difference overlay.

[\[comparison\]](#) [\[QA\]](#) [\[A/B\]](#) [\[wipe\]](#) [\[blink\]](#)

Algorithm: Shared viewport model, synchronised scroll/zoom, difference image rendering

Blind Deconvolution

[blind_deconv.py](#)

Sharpens images by estimating and removing the Point Spread Function without a separately measured PSF input. Uses regularised Wiener-Hunt deconvolution with automatic PSF estimation from bright stars in the field.

[\[deconvolution\]](#) [\[sharpening\]](#) [\[PSF\]](#) [\[Wiener\]](#)

Algorithm: Regularised Wiener-Hunt deblurring, Richardson-Lucy iterations, automatic PSF estimation

Pattern Noise Corrector

[pattern_noise_corrector.py](#)

Identifies and removes fixed-pattern noise artifacts common in CMOS sensors including amplifier glow, column banding and horizontal row striping. The Münch 2009 wavelet-Fourier method cleanly separates stripe noise from real signal in frequency space.

[\[CMOS\]](#) [\[pattern-noise\]](#) [\[banding\]](#) [\[amp-glow\]](#) [\[destripe\]](#)

Algorithm: Wavelet-Fourier stripe filter (Münch et al. 2009), column/row frequency isolation

Reference: Münch et al. 2009, Optics Express

Stacking Suite

[stacking_suite.py](#)

Combines multiple calibrated exposures into a single deep image through statistical rejection and averaging. Covers subframe quality ranking, advanced sigma-clipping stacking, drizzle resampling for undersampled sensors, and multi-night session management with automatic alignment and offset correction.

Vectorised Winsorized Sigma Stack

[VectorisedWinsorizedSigmaStack.py](#)

High-performance sigma-clipping stacker using Winsorized statistics for robust outlier handling. Instead of discarding rejected pixels, Winsorized clipping replaces them with the nearest non-rejected boundary value, preserving the noise characteristics of the stack while eliminating hot pixels, cosmic rays and satellite trails.

[\[stacking\]](#) [\[sigma-clipping\]](#) [\[Winsorized\]](#) [\[rejection\]](#)

Algorithm: Vectorised Winsorized sigma clipping, iterative kappa-sigma boundary estimation, numpy batch processing

Subframe Ranking

[subframe_ranking.py](#)

Automatically grades every subframe in a sequence by measuring FWHM, eccentricity, signal-to-noise ratio and sky background level. Outputs a ranked list with configurable quality thresholds to select the best N% of frames before stacking.

[\[quality\]](#) [\[FWHM\]](#) [\[eccentricity\]](#) [\[SNR\]](#) [\[ranking\]](#) [\[selection\]](#)

Algorithm: PSF fitting for FWHM/eccentricity, aperture photometry for SNR, sigma-clipped sky estimation

Drizzle Combiner

`drizzle_combiner.py`

Applies the drizzle algorithm to combine dithered subframes at higher than native resolution. Particularly valuable for undersampled optical systems where the PSF is smaller than a pixel. Supports configurable pixfrac (0.1–1.0) and output scale factors from 0.5× to 2×.

`[drizzle] [resampling] [resolution] [dithering] [pixfrac]`

Algorithm: Drizzle interpolation (Fruchter & Hook 2002), sub-pixel weighting, bad pixel propagation

Reference: Fruchter & Hook 2002, PASP 114, 144

Multi-Session Manager

`multi_session_manager.py`

Aligns and combines FITS sequences acquired across multiple nights or with different instruments. Automatically measures frame-to-frame offsets using astrometry or cross-correlation, applies photometric normalisation to match background levels, and produces a combined master stack.

`[multi-night] [session] [alignment] [mosaic] [normalisation]`

Algorithm: Cross-correlation or WCS-based alignment, iterative photometric scaling, weighted co-addition

Lucky & Restoration Suite

`lucky_suite.py`

High-resolution imaging suite for planetary, binary star and fine-detail astrophotography. Combines lucky imaging frame selection with advanced speckle reconstruction and deep-learning-based restoration methods. Achieves diffraction-limited resolution from ground-based observations by exploiting moments when atmospheric turbulence is momentarily low.

Lucky Preprocessor

`lucky_preprocessor.py`

Selects the sharpest N% of video frames from a planetary or binary star sequence by measuring the Laplacian energy (high-frequency content) of each frame. Frames with the highest scores contain the best-preserved diffraction-limited detail.

`[lucky] [frame-selection] [Laplacian] [planetary]`

Algorithm: Laplacian energy quality metric, configurable percentile threshold, batch FITS/SER processing

Bispectrum Speckle

`bispectrum_speckle.py`

Achieves diffraction-limited resolution through bispectrum analysis of speckle images. The bispectrum preserves phase information that is lost in the power spectrum alone, enabling full image reconstruction from short-exposure speckle frames without needing a reference star in all cases.

`[speckle] [bispectrum] [diffraction-limited] [phase-recovery]`

Algorithm: Triple-correlation bispectrum, Fourier phase reconstruction, Knox-Thompson variant

Reference: Lohmann, Weigelt & Wirtitzer 1983, Applied Optics

Speckle Holography

`speckle_holography.py`

Reconstructs diffraction-limited images of binary stars and compact objects using the speckle holography technique. Uses a reference star observed in the same isoplanatic patch to calibrate the atmospheric transfer function and deconvolve the object speckle pattern.

`[speckle] [holography] [binary-stars] [deconvolution]`

Algorithm: Cross-spectrum analysis, isoplanatic reference deconvolution (Schödel et al. 2013)

Reference: Schödel et al. 2013, A&A; 558, A33

Thresher Stack

[thresher_stack.py](#)

Implements The Thresher algorithm for online multi-frame blind deconvolution. Unlike lucky imaging, Thresher uses ALL frames rather than discarding poor ones — the information in blurred frames is still exploited through a maximum-likelihood image model.

[\[thresher\]](#) [\[blind-deconv\]](#) [\[all-frames\]](#) [\[multi-frame\]](#)

Algorithm: Expectation-Maximisation blind deconvolution (Hitchcock et al. 2022, MNRAS 511)

Reference: Hitchcock et al. 2022, MNRAS 511, 5372

DIPLI

[dipli.py](#)

Deep Image Prior Lucky Imaging using a 4-stage U-Net architecture with Stochastic Gradient Langevin Dynamics (SGLD) Bayesian inference. Learns the image structure from a small number of best-selected frames (7–13) and iteratively refines the reconstruction. MC averaging of the final 10% of SGLD iterates reduces noise amplification.

[\[deep-learning\]](#) [\[U-Net\]](#) [\[SGLD\]](#) [\[Bayesian\]](#) [\[lucky\]](#)

Algorithm: 4-stage U-Net + SGLD inference, TV-L1 optical flow confidence maps, Singh et al. 2025

Reference: Singh et al. 2025, arXiv:2503.15984

ImageMM

[imagemm.py](#)

Multi-frame deconvolution using matched filtering and maximum marginalisation to jointly estimate the object and PSF from a set of frames with different blur realisations. Well-suited for planetary sequences where the seeing varies rapidly between frames.

[\[multi-frame\]](#) [\[deconvolution\]](#) [\[matched-filter\]](#) [\[marginalisation\]](#)

Algorithm: Maximum marginalisation PSF estimation, matched filter initialisation, iterative EM updates

Phase Diversity Wavefront Sensing

[phase_diversity_wavefront.py](#)

Reconstructs the optical wavefront aberrations of the telescope from a pair of in-focus and deliberately defocused images of the same object. The phase diversity between the two images encodes the aberration map, which can then be used to correct optical alignment or characterise seeing.

[\[wavefront\]](#) [\[aberration\]](#) [\[phase-diversity\]](#) [\[optics\]](#)

Algorithm: Phase diversity optimisation, Zernike modal decomposition, gradient-based wavefront retrieval

ADI KLIP

[adi_klip.py](#)

Angular Differential Imaging combined with the Karhunen-Loève Image Processing (KLIP) algorithm for detecting faint companions such as exoplanets and circumstellar discs. The telescope rotation during an ADI sequence allows the stellar PSF halo to be modelled and subtracted while preserving off-axis signal.

[\[ADI\]](#) [\[KLIP\]](#) [\[exoplanet\]](#) [\[companion\]](#) [\[coronagraphy\]](#)

Algorithm: ADI frame subtraction, KL basis construction, KLIP projection (Soummer et al. 2012)

Reference: Soummer, Pueyo & Carlotti 2012, ApJ 755, L28

Solar System Suite

[solar_system_suite.py](#)

End-to-end pipeline for all moving solar system objects: comets, asteroids, planets and moons. Covers dual-stack comet imaging, moving object tracklet generation and MPC submission formatting, planetary surface texture mapping, field rotation correction and full Keplerian orbit determination from multi-epoch astrometry.

Comet Pipeline

`comet_pipeline.py`

Produces two optimally-combined stacks from a comet imaging sequence: one aligned on the background stars, and one aligned on the cometary nucleus. Both stacks are merged to show the coma and nucleus together against a sharp stellar background using the nucleus-tracked image as a mask.

`[comet] [dual-stack] [nucleus-tracking] [coma]`

Algorithm: Star-aligned and comet-aligned sigma stacks, nucleus centroid tracking, weighted mask merge

Comet Coma Analyser

`comet_coma_analyser.py`

Characterises the cometary coma through surface brightness profile fitting. Measures the Af(rho) dust production parameter, radial falloff exponent, coma asymmetry index and tail position angle from multi-epoch observations to track activity evolution.

`[comet] [coma] [Af-rho] [photometry] [activity]`

Algorithm: Elliptical aperture photometry, power-law surface brightness fitting, Af(rho) from A'Hearn et al.

Reference: A'Hearn et al. 1984, AJ 89, 579

Moving Object Detector

`moving_object.py`

Detects asteroids, comets and other solar system objects moving against the background stars across a sequence of aligned FITS frames. Generates tracklet files in Minor Planet Center (MPC) format for official reporting and orbit determination.

`[asteroid] [MPC] [tracklets] [moving-objects] [detection]`

Algorithm: Difference image source detection, linear motion linking, MPC 80-column format output

Planet Derotation

`planet_derotation.py`

Corrects field rotation in planetary video sequences acquired with alt-azimuth mounted telescopes. Computes the parallactic angle for each frame from the observer's coordinates and the object's hour angle, then derotates each frame to a common orientation before stacking.

`[planetary] [derotation] [alt-az] [parallactic-angle]`

Algorithm: Parallactic angle calculation from observer HA/Dec, bilinear rotation interpolation

Planetary Texture Mapper

`planetary_texture_mapper.py`

Projects resolved planetary disk images onto a cylindrical equirectangular coordinate system to build a full-surface texture map from multiple observations. Accounts for limb darkening, the sub-observer latitude, system III longitude and combines data from multiple sessions for 360-degree coverage.

`[planetary] [texture] [projection] [cylindrical] [limb-darkening]`

Algorithm: Orthographic-to-equirectangular reprojection, limb darkening polynomial correction, multi-session co-registration

Orbit Determinator

`orbit_determinator.py`

Computes a preliminary Keplerian orbit from two or more MPC-format astrometric observations using the Gauss method for initial orbit determination. Refines the solution by differential correction and estimates orbital element uncertainties through Monte Carlo propagation of observational errors.

`[orbit] [Keplerian] [Gauss-method] [MPC] [Monte-Carlo]`

Algorithm: Gauss method IOD, differential correction, MC uncertainty propagation (Bate, Mueller & White)

Reference: Bate, Mueller & White 1971, Fundamentals of Astrodynamics

Variable Star Suite

`variable_star_suite.py`

Complete differential photometry pipeline for variable stars, exoplanet transits and stellar occultations. Implements the Broeg optimal comparison star weighting scheme, Lomb-Scargle and wavelet period analysis, full MCMC exoplanet transit fitting and high-precision occultation chord timing. Results are formatted for direct AAVSO and VSX export.

Variable Star Pipeline

`variable_star.py`

End-to-end variable star differential photometry pipeline. Performs aperture photometry on all sources in a field sequence, selects stable comparison stars, computes differential light curves, cross-matches with the VSX catalogue and exports observations in AAVSO extended format.

`[variable-star] [photometry] [AAVSO] [VSX] [light-curve]`

Algorithm: Aperture photometry, comparison star selection, differential magnitude, AAVSO extended format

Broeg Photometry

`broeg_photometry.py`

Implements the Broeg 2005 algorithm for constructing an optimally-weighted artificial comparison star from an ensemble of reference stars. Each comparison star is weighted by the inverse square of its scatter ($1/\sigma^2$), iteratively refined until convergence.

`[photometry] [Broeg] [optimal-weighting] [comparison-star]`

Algorithm: Iterative $1/\sigma^2$ weighting convergence (Broeg et al. 2005)

Reference: Broeg, Fernandez & Neuhäuser 2005, AN 326, 134

Extinction Map

`extinction_map.py`

Derives the nightly atmospheric extinction coefficient by fitting standard star magnitudes observed at multiple airmass values throughout the night. The extinction map can be applied to correct target photometry to outside-atmosphere magnitudes.

`[extinction] [airmass] [standard-stars] [atmosphere]`

Algorithm: Bouguer line fitting (mag vs. airmass), sigma-clipped least squares, colour term estimation

Period Finder

`period_finder.py`

Multi-method period analysis suite for variable star and exoplanet light curves. Combines the Lomb-Scargle periodogram for unevenly-sampled data with the Autocorrelation Function and wavelet scalogram methods, using cross-validation to select the most likely period.

`[period] [Lomb-Scargle] [ACF] [wavelet] [periodogram]`

Algorithm: Lomb-Scargle periodogram (Scargle 1982), ACF, continuous wavelet transform, cross-validation

Reference: Scargle 1982, ApJ 263, 835; VanderPlas 2018, ApJS 236, 16

Transit MCMC

`transit_mcmc.py`

Fits a transit model to an exoplanet or eclipsing binary light curve using Markov Chain Monte Carlo sampling to estimate the posterior distributions of all transit parameters: the radius ratio R_p/R_s , impact parameter b , semi-major axis a/R_s , and limb darkening coefficients.

`[transit] [MCMC] [exoplanet] [Rp/Rs] [limb-darkening]`

Algorithm: Mandel-Agol transit model, quadratic limb darkening (Claret 2000), emcee MCMC sampler

Reference: Mandel & Agol 2002, ApJ 580, L171; Claret 2000, A&A; 363, 1081

Occultation Timing

`occultation_timing.py`

Provides high-precision ingress and egress timing of stellar occultations by solar system bodies. Fits a square-wave occultation model to the high-cadence light curve, derives the chord length across the occulting body, and estimates its size when combined with multiple-station chords.

`[occultation] [timing] [asteroid] [chord] [high-cadence]`

Algorithm: Square-wave model fitting, Bayesian timing uncertainty estimation, multi-chord geometry

Transient Pipeline

`transient_pipeline.py`

Automated image subtraction and transient detection pipeline for discovering supernovae, novae, variable nuclei and fast transients. Implements two independent difference imaging methods — ZOGY and SFFT — followed by automated source detection and rule-based real/bogus classification. Designed for systematic multi-epoch survey programmes.

ZOGY Subtraction

`zogy_subtract.py`

Optimal image subtraction in Fourier space using the ZOGY method. Correctly propagates noise through the subtraction, producing a statistically optimal difference image (D) and a PSF-matched significance image (Scorr) where detected sources have well-defined statistical significance.

`[ZOGY] [subtraction] [transient] [Fourier] [Scorr]`

Algorithm: ZOGY Fourier-domain subtraction with proper noise propagation (Zackay, Ofek & Gal-Yam 2016)

Reference: Zackay, Ofek & Gal-Yam 2016, ApJ 830, 27

SFFT Subtraction

`sfft_subtract.py`

Saccadic Fast Fourier Transform image subtraction using tiled kernel estimation. Solves for a spatially varying convolution kernel in overlapping tiles and blends them with Hann-window apodisation. Scales as $O(N \log N)$ making it practical for large 16+ Mpx sensors.

`[SFFT] [subtraction] [tiled] [spatially-varying] [O(NlogN)]`

Algorithm: Tiled kernel estimation, Hann-window tile blending (Hu et al. 2022, arXiv:2109.09334)

Reference: Hu et al. 2022, arXiv:2109.09334

Transient Detector

`transient_detector.py`

Detects point and extended sources in difference images produced by ZOGY or SFFT subtraction. Uses DAOSTarFinder with adaptive threshold based on the local noise map, cross-matches detections with known catalogued sources to flag known variables, and outputs a candidate table.

`[detection] [DAOSTarFinder] [difference-image] [catalogue]`

Algorithm: DAOSTarFinder with local RMS threshold, catalogue cross-match (Gaia DR3, VSX), candidate table

Transient Vetter

`transient_vetter.py`

Scores each transient candidate on a 0–100 real/bogus scale using a set of hand-crafted discriminating features: PSF consistency with the reference star profile, morphological regularity, proximity to image borders, pixel saturation, and difference image local context.

`[vetting] [real/bogus] [classifier] [PSF] [morphology]`

Algorithm: Rule-based feature scoring: PSF chi-squared, ellipticity, border proximity, local S/N

Session QA Suite

`session_qa_suite.py`

Session quality analysis tools for characterising the optical and atmospheric conditions during an imaging run. Measures atmospheric seeing, maps PSF variation across the field, charts astrometric distortion patterns, detects proper motion between epochs, and provides A/B frame comparison. Results guide decisions about which nights and frames are worth including in the final stack.

Seeing Estimator

`seeing_estimator.py`

Estimates the atmospheric seeing from the FWHM of unsaturated stars measured across multiple frames and airmass values. Fits the FWHM vs. airmass relation to derive the zenith seeing and the Fried coherence length r_0 , a standard measure of atmospheric turbulence strength.

`[seeing] [FWHM] [r0] [Fried] [airmass] [atmosphere]`

Algorithm: Gaussian PSF fitting, FWHM vs. $\text{airmass}^{0.6}$ power-law fit, Fried r_0 from $\lambda/(\text{FWHM at zenith})$

PSF Heatmap

`psf_heatmap.py`

Maps the spatial variation of PSF FWHM and ellipticity across the entire imaging field by measuring stars in a grid of positions. The resulting heatmap reveals optical aberrations such as field curvature, coma and astigmatism at the field edges, and informs the spatially varying PSF deconvolution script.

`[PSF] [FWHM] [ellipticity] [field-variation] [heatmap] [optics]`

Algorithm: Grid-based star PSF fitting, ellipticity from second-moment matrix, spatial interpolation

Astrometric Distortion Mapper

`astrometric_distortion_mapper.py`

Quantifies the astrometric distortion pattern of the optical system by comparing measured star positions to their catalogue coordinates from Gaia DR3. Fits polynomial and radial distortion models to the residuals to characterise barrel/pincushion distortion and decentring.

`[astrometry] [WCS] [distortion] [Gaia] [polynomial]`

Algorithm: WCS residual analysis, polynomial + radial distortion model fit, Gaia DR3 cross-match

Proper Motion Finder

`proper_motion_finder.py`

Detects stars with measurable proper motion by comparing astrometric positions across multi-epoch observations separated by months to years. Cross-matches detections with the SIMBAD database to identify known high proper motion stars and flags potential new candidates.

`[proper-motion] [astrometry] [multi-epoch] [SIMBAD]`

Algorithm: Epoch-to-epoch astrometric differencing, proper motion vector fitting, SIMBAD cross-match

Spectroscopy Suite

`spectroscopy_suite.py`

Amateur spectroscopy pipeline covering the full workflow from raw 2D spectral image to calibrated 1D spectrum and scientific measurements. Handles CCD fringe correction which is a major nuisance in back-illuminated red-sensitive sensors, performs optimal 1D extraction and wavelength calibration, and measures radial velocities through cross-correlation with stellar template spectra.

Spectral Extractor

`spectral_extractor.py`

Extracts a 1D spectrum from a 2D spectral image using the Horne 1986 optimal extraction algorithm, which weights each pixel by its signal-to-noise ratio to maximise the quality of the extracted spectrum. Performs wavelength calibration from arc lamp spectra and flat-field correction.

`[spectroscopy] [1d-extraction] [optimal] [wavelength-cal] [flat]`

Algorithm: Horne 1986 optimal extraction, arc lamp wavelength calibration, flat-field correction

Reference: Horne 1986, PASP 98, 609

CCD Fringe Corrector

[ccd_fringe_corrector.py](#)

Removes interference fringe patterns from back-illuminated CCD spectral images using a physics-based Fabry-Pérot etalon model. The fringe Optical Path Difference is detected via FFT on a wavenumber grid, and the fringe amplitude envelope is measured using a Hilbert transform before subtraction.

[CCD] [fringes] [spectroscopy] [Fabry-Pérot] [OPD] [Hilbert]

Algorithm: FFT OPD detection on wavenumber grid, Hilbert amplitude envelope, etalon model subtraction

Radial Velocity

[radial_velocity.py](#)

Measures stellar radial velocities by cross-correlating the observed spectrum with a set of synthetic stellar templates spanning spectral types G2V, K0V, M2V and A0V. Fits a Gaussian to the cross-correlation function peak to determine the Doppler shift and its uncertainty.

[radial-velocity] [CCF] [Doppler] [stellar-template] [spectroscopy]

Algorithm: CCF with G2V/K0V/M2V/A0V templates (Kurucz/PHOENIX), Gaussian bisector fit, RV uncertainty from CCF width

Standalone Scripts

[Scripts/](#)

Additional standalone scripts covering specialised calibration, sensor characterisation, photometry and solar system tasks. These scripts operate independently and can also be used within the main suite pipelines.

Bias Drift Corrector

[bias_drift_corrector.py](#)

Corrects for thermal drift in the bias level over long imaging sessions by measuring the overscan region or pre-defined bias columns in each frame and subtracting the time-varying offset.

[bias] [drift] [calibration] [thermal]

Algorithm: Overscan/bias column median tracking, polynomial drift model, per-frame correction

Cosmic Ray Rejection

[cosmic_ray_rejection.py](#)

Detects and masks cosmic ray hits in single FITS frames using the L.A.Cosmic algorithm based on Laplacian edge detection. Replaces cosmic ray pixels with interpolated values from surrounding clean pixels.

[cosmic-rays] [L.A.Cosmic] [Laplacian] [single-frame]

Algorithm: L.A.Cosmic Laplacian edge detection (van Dokkum 2001)

Reference: van Dokkum 2001, PASP 113, 1420

Dark Temperature Corrector

[dark_temp_corrector.py](#)

Scales a master dark frame to match a different sensor temperature by applying the dark current doubling rule. The dark current in silicon sensors doubles approximately every 5.5-6°C, allowing a single temperature master dark to be corrected for use at other temperatures.

[dark] [temperature] [calibration] [doubling-rule]

Algorithm: Dark current doubling-rule scaling: $D(T_2) = D(T_1) * 2^{(T_2-T_1)/T_d}$

PSF Photometry

[psf_photometry.py](#)

Performs point-spread-function fitting photometry for crowded stellar fields where aperture photometry suffers from source blending. Builds an empirical PSF model from isolated bright stars and fits it to every source in the field simultaneously.

[photometry] [PSF-fitting] [crowded] [blending]

Algorithm: Empirical PSF construction (photutils EPSFBuilder), simultaneous multi-source fitting

Mosaic Photometric Matching

[mosaic_photometric_matching.py](#)

Photometrically matches overlapping mosaic panels by measuring common stars in the overlap regions and deriving multiplicative and additive correction factors to bring all panels to a uniform brightness scale.

[\[mosaic\]](#) [\[photometry\]](#) [\[matching\]](#) [\[normalisation\]](#)

Algorithm: Overlap region star matching, least-squares multiplicative/additive correction matrix

Planetary Zone Stacker

[planetary_zone_stacker.py](#)

Applies zone-based selective stacking to planetary images, dividing the disk into spatial zones and optimising the stacking independently for each zone based on local quality metrics. Brings out detail simultaneously in the bright equatorial bands and faint polar regions.

[\[planetary\]](#) [\[zone-stacking\]](#) [\[local-quality\]](#) [\[adaptive\]](#)

Algorithm: Disk zone segmentation, per-zone Laplacian quality metric, weighted blend

Satellite Trail Remover

[satellite_trail_remover.py](#)

Detects and removes satellite, aircraft and meteor trails from individual FITS frames. Uses Hough transform line detection on a gradient image to identify linear features, then inpaints the trail region using surrounding pixel statistics.

[\[satellite\]](#) [\[trail\]](#) [\[Hough\]](#) [\[inpainting\]](#) [\[artefact\]](#)

Algorithm: Hough transform line detection, morphological trail mask, inpainting from local statistics

Software ADC

[software_adc.py](#)

Corrects for atmospheric dispersion in planetary images by measuring the colour-dependent shift between red and blue wavelengths and applying sub-pixel channel registration to align them. Particularly important for low-elevation observations where atmospheric refraction is strongest.

[\[ADC\]](#) [\[dispersion\]](#) [\[planetary\]](#) [\[colour-shift\]](#) [\[refraction\]](#)

Algorithm: Per-channel cross-correlation shift, sub-pixel Fourier interpolation, elevation-dependent model

Background Gradient

[bg_gradient.py](#)

Models and subtracts the sky background gradient from deep-sky images. Generates a smooth background model by sampling source-free sky regions, fitting a 2D polynomial or spline surface, and subtracting it from the image.

[\[background\]](#) [\[gradient\]](#) [\[sky\]](#) [\[polynomial\]](#) [\[spline\]](#)

Algorithm: Source-masked background sampling, 2D polynomial/spline surface fit

Exoplanet Transit Suite

[exoplanet_transit_suite.py](#)

Dedicated suite for exoplanet transit observation, combining differential photometry, systematic detrending and transit model fitting in a single workflow optimised for the output of a typical single-night transit observation.

[\[exoplanet\]](#) [\[transit\]](#) [\[detrending\]](#) [\[photometry\]](#)

Algorithm: Aperture photometry, airmass/systematics detrending, Mandel-Agol transit fit

SYSREM Detrend

[sysrem_detrend.py](#)

Applies the SYSREM algorithm for systematic trend removal from ensemble light curves. Iteratively identifies and removes common-mode systematics (atmospheric, instrumental) shared across multiple comparison stars without removing astrophysical signals.

[\[SYSREM\]](#) [\[detrending\]](#) [\[systematics\]](#) [\[ensemble\]](#) [\[light-curve\]](#)

Algorithm: SYSREM iterative common-mode removal (Tamuz et al. 2005), transit window masking

Reference: Tamuz, Mazeh & Zucker 2005, MNRAS 356, 1466

Sersic Profiler

`siersic_profiler.py`

Fits the Sersic surface brightness profile to galaxy images to extract morphological parameters: the effective radius R_e , Sersic index n and surface brightness I_e . PSF-convolved fitting recovers intrinsic parameters even for partially resolved galaxies. Also performs Jedrzejewski 1987 isophote fitting to measure ellipticity and position angle radial profiles.

`[Sersic] [galaxy] [morphology] [re] [isophote] [ellipticity]`

Algorithm: PSF-convolved 1D Sersic fit (Ciotti & Bertin 1999), Nelder-Mead optimisation, Jedrzejewski isophote fitting

Reference: Ciotti & Bertin 1999, A&A; 352, 447; Jedrzejewski 1987, MNRAS 226, 747

Spatially Varying PSF Deconvolution

`spatially_varying_psf_deconv.py`

Position-dependent Richardson-Lucy deconvolution that uses the PSF FWHM map from `psf_heatmap.py` to apply a locally-appropriate deconvolution to each region of the image. Each tile uses the median PSF FWHM measured in that region; tiles are blended with Hann-window apodisation to avoid seams.

`[deconvolution] [spatially-varying] [RL] [PSF-map] [tiled]`

Algorithm: Tiled Richardson-Lucy with local Gaussian PSF, Hann-window tile blending

Reference: Lucy 1974, AJ 79, 745; Richardson 1972, JOSA 62, 55

Equipment Manager

`equipment_manager.py`

Central equipment profile manager and shared settings store for the entire HYPERLOAD suite. Stores telescope, camera, filter and site information that all other scripts can read. Acts as the dependency root — imported by all suite scripts to avoid re-entering equipment parameters for every tool.

`[equipment] [profile] [settings] [telescope] [camera]`

Algorithm: JSON-based profile persistence, shared settings API, per-script JSON fallback

HYPERLOAD Launcher

The launcher is the single entry point that Siril executes. It provides a unified graphical workspace for discovering, loading, configuring and running all 67+ scripts in the collection.

Suite tree sidebar

All suites and their member scripts are listed in a collapsible tree. Click a suite header to expand it. Click a script row to open its floating panel. Each suite can be individually loaded or unloaded from memory using the toggle button.

Floating script panels

Each script opens as a floating panel in the desktop workspace. Panels can be dragged by their title bar, resized from any of 8 edge/corner handles, and collapsed to just the title bar by double-clicking. Multiple panels can be open simultaneously.

File dock

A horizontal file dock at the bottom of the window holds files to be processed. Drag and drop files from the dock onto a script panel or directly onto a script row in the tree to attach them to that script. Use the Add files button to load files from disk.

Live system statistics

The top bar shows real-time CPU usage, RAM consumption and disk usage with animated progress bars. The status LED indicates whether any script is currently running.

Path management

Working directory, import path and export path are set once in the sidebar and automatically propagated as default paths to every script panel. Settings are saved to `hyperload_config.json` between sessions.

Crash detection

If a launched script crashes within 3 seconds, the launcher reads the `stderr` output from the process and displays the Python traceback in an error dialog, allowing rapid diagnosis without inspecting Siril's log.

Quick Reference — All Scripts

Script file	Suite	Primary function
wavelet_stretch.py	Image Processing Suite	Multi-scale wavelet-based non-linear stretching that preserves fine detail and colour accuracy simultaneously.
dark_channel.py	Image Processing Suite	Removes atmospheric haze, gradient vignetting and sky glow using the dark channel prior from computational photography.
mask_builder.py	Image Processing Suite	Constructs pixel-precise masks for stars, galaxies and nebulae using source detection and morphological operations.
nb_palette.py	Image Processing Suite	Maps narrowband emission line data (H-alpha, OIII, SII) to RGB colour space using configurable palette presets.
local_norm.py	Image Processing Suite	Removes large-scale background gradient variations by fitting and subtracting a smooth background model derived from source-free image regions.
ab_compare.py	Image Processing Suite	Side-by-side comparator for evaluating two processing variants.
blind_deconv.py	Image Processing Suite	Sharpens images by estimating and removing the Point Spread Function without a separately measured PSF input.
pattern_noise_corrector.py	Image Processing Suite	Identifies and removes fixed-pattern noise artifacts common in CMOS sensors including amplifier glow, column banding and horizontal row striping.
VectorisedWinsorizedSigmaStack.py	Stacking Suite	High-performance sigma-clipping stacker using Winsorized statistics for robust outlier handling.
subframe_ranking.py	Stacking Suite	Automatically grades every subframe in a sequence by measuring FWHM, eccentricity, signal-to-noise ratio and sky background level.
drizzle_combiner.py	Stacking Suite	Applies the drizzle algorithm to combine dithered subframes at higher than native resolution.
multi_session_manager.py	Stacking Suite	Aligns and combines FITS sequences acquired across multiple nights or with different instruments.
lucky_preprocessor.py	Lucky & Restoration Suite	Selects the sharpest N% of video frames from a planetary or binary star sequence by measuring the Laplacian energy (high-frequency content) of each frame.
bispectrum_speckle.py	Lucky & Restoration Suite	Achieves diffraction-limited resolution through bispectrum analysis of speckle images.
speckle_holography.py	Lucky & Restoration Suite	Reconstructs diffraction-limited images of binary stars and compact objects using the speckle holography technique.
thresher_stack.py	Lucky & Restoration Suite	Implements The Thresher algorithm for online multi-frame blind deconvolution.

